

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 11

Name: S Svava Haasini

Roll No: 2520090170

Task 1: Module Integration and Interface Validation

Question:

Connect modules, validate interfaces, test end-to-end execution, handle errors, improve reliability, and refactor components.

Source Code:

```
GNU nano 7.2                                operations.h *
```

```
#ifndef OPERATIONS_H
#define OPERATIONS_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
GNU nano 7.2                                operations.c *
```

```
#include "operations.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}
```

```

GNU nano 7.2                                T1.c *
#include <stdio.h>
#include "operations.h"

int main(void)
{
    int a = 20;
    int b = 8;

    printf("=== Module Integration Test ===\n");

    printf("Input values: a = %d, b = %d\n", a, b);

    printf("Addition result      : %d\n", add(a, b));
    printf("Subtraction result   : %d\n", subtract(a, b));

    printf("\nModule interfaces validated successfully.\n");
    printf("End-to-end execution completed.\n");

    return 0;
}

```

Output:

```

/home/haasini/OSSP_upload/Skill_Week11
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano operations.h
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano operations.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra T1.c operations.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra T1.c operations.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week11$ ./T1
=== Module Integration Test ===
Input values: a = 20, b = 8
Addition result      : 28
Subtraction result   : 12

Module interfaces validated successfully.
End-to-end execution completed.
haasini@SSH:~/OSSP_upload/Skill_Week11$ |

```

Observations:

1. A header file (operations.h) defines function prototypes for add() and subtract() with function declarations protected by #ifndef guards.
2. An implementation file (operations.c) provides the actual function bodies for add() and subtract() operations on integer operands.
3. The add() function returns the sum of two integers (a + b), implementing basic arithmetic addition.
4. The subtract() function returns the difference of two integers (a - b), implementing basic arithmetic subtraction.
5. A test program (T1.c) includes both the header file and stdio.h to access the arithmetic functions and standard I/O capabilities.
6. The test initializes two integer values (a=20, b=8) and calls both add() and subtract() with these values.

7. The output displays 'Module Integration Test', input values (a=20, b=8), and computed results: Addition=28, Subtraction=12.
8. Success messages 'Module interfaces validated successfully' and 'End-to-end execution completed' confirm proper module connectivity and functionality.
9. The integration demonstrates compile-time linking of multiple source files and successful runtime execution of modular components.

Task 2: Runtime Error Detection and Logging

Question:

Detect runtime errors, generate error messages, log failures, handle invalid syntax, recover gracefully, and test error scenarios.

Source Code:

```
GNU nano 7.2 T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    FILE *log;
    log = fopen("error.log", "a");
    if (log == NULL)
    {
        perror("error.log");
        return 1;
    }

    if (argc != 2)
    {
        printf("Error: invalid syntax.\n");
        printf("Usage: %s <filename>\n", argv[0]);
        fprintf(log,
            "Invalid syntax: expected one filename argument.\n");
        fclose(log);
        printf("Error logged. Program recovered gracefully.\n");
        return 1;
    }

    FILE *file = fopen(argv[1], "r");
    if (file == NULL)
    {
        printf("Runtime error: cannot open '%s'.\n", argv[1]);
        printf("Reason: %s\n", strerror(errno));
        fprintf(log,
            "Failed to open '%s': %s\n",
            argv[1],
            strerror(errno));
        fclose(log);
        printf("Failure logged to error.log.\n");
        printf("Program recovered gracefully.\n");
        return 1;
    }

    printf("File '%s' opened successfully.\n", argv[1]);
    fclose(file);
    fclose(log);
    printf("No runtime error detected.\n");
    return 0;
}
```

Output:

```
End to end execution completed.
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano T2.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra T2.c -o T2
haasini@SSH:~/OSSP_upload/Skill_Week11$ ./T2
Error: invalid syntax.
Usage: ./T2 <filename>
Error logged. Program recovered gracefully.
haasini@SSH:~/OSSP_upload/Skill_Week11$ ./T2 missing_file.txt
Runtime error: cannot open 'missing_file.txt'.
Reason: No such file or directory
Failure logged to error.log.
Program recovered gracefully.
haasini@SSH:~/OSSP_upload/Skill_Week11$ cat error.log
Invalid syntax: expected one filename argument.
Failed to open 'missing_file.txt': No such file or directory
haasini@SSH:~/OSSP_upload/Skill_Week11$ |
```

Observations:

1. A log file (error.log) is opened in append mode ('a') at the start of the program, accumulating error messages across multiple executions.
2. If error.log itself cannot be opened, perror("error.log") reports that specific failure; for all other errors, strerror(errno) provides the failure reason.
3. Command-line argument validation checks argc != 2, requiring exactly one filename argument for proper program usage.
4. When argc validation fails, the program prints an invalid-syntax error message to stdout, logs the failure to error.log, closes the log, and returns 1.
5. If the requested filename cannot be opened, strerror(errno) retrieves and displays the operating-system reason for the failure.
6. fprintf() writes formatted error messages to error.log, capturing the failure description for later review.
7. fclose() closes both the log file and any successfully opened file handle, ensuring proper resource cleanup before exit.
8. The program recovers gracefully from every error by printing a recovery message and returning without crashing.
9. The supplied output demonstrates exactly two tested error scenarios: running './T2' with no filename argument (invalid syntax), and running './T2 missing_file.txt' (file-open failure). The error.log records both the invalid-syntax and missing-file failures shown in the output.

Task 3: Memory Leak Detection and Fix Verification

Question:

Run Valgrind, identify leaks, fix memory errors, release resources, verify cleanup, and document findings.

Source Code:

```
GNU nano 7.2 T3_leak.c *
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *numbers = malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        numbers[i] = (i + 1) * 10;

    printf("Allocated values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", numbers[i]);

    printf("\n");

    printf("Memory intentionally not freed for leak test.\n");

    return 0;
}
```

Valgrind Detection (Before Fix):

```
haasini@SSH:~$ cd ~/OSSP_upload
mkdir -p Skill_Week11
cd Skill_Week11
pwd
/home/haasini/OSSP_upload/Skill_Week11
haasini@SSH:~/OSSP_upload/Skill_Week11$ valgrind --version
valgrind-3.22.0
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano T3_leak.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra -g T3_leak.c -o T3_leak
haasini@SSH:~/OSSP_upload/Skill_Week11$ valgrind --leak-check=full ./T3_leak
==507== Memcheck, a memory error detector
==507== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==507== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==507== Command: ./T3_leak
==507==
Allocated values: 10 20 30 40 50
Memory intentionally not freed for leak test.
==507==
==507== HEAP SUMMARY:
==507==     in use at exit: 20 bytes in 1 blocks
==507==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==507==
==507== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==507==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==507==    by 0x1091BE: main (T3_leak.c:6)
==507==
==507== LEAK SUMMARY:
==507==     definitely lost: 20 bytes in 1 blocks
==507==     indirectly lost: 0 bytes in 0 blocks
==507==     possibly lost: 0 bytes in 0 blocks
==507==     still reachable: 0 bytes in 0 blocks
==507==     suppressed: 0 bytes in 0 blocks
==507==
==507== For lists of detected and suppressed errors, rerun with: -s
==507== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
haasini@SSH:~/OSSP_upload/Skill_Week11$ |
```

```

GNU nano 7.2                                T3_fixed.c *
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *numbers = malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        numbers[i] = (i + 1) * 10;

    printf("Allocated values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", numbers[i]);

    printf("\n");

    free(numbers);

    printf("Allocated memory released using free().\n");

    return 0;
}

```

Valgrind Verification (After Fix):

```

valgrind: ./T3_fixed: no such file or directory
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano T3_fixed.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra -g T3_fixed.c -o T3_fixed
haasini@SSH:~/OSSP_upload/Skill_Week11$ valgrind --leak-check=full ./T3_fixed
==631== Memcheck, a memory error detector
==631== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==631== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==631== Command: ./T3_fixed
==631==
Allocated values: 10 20 30 40 50
Allocated memory released using free().
==631==
==631== HEAP SUMMARY:
==631==   in use at exit: 0 bytes in 0 blocks
==631==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==631==
==631== All heap blocks were freed -- no leaks are possible
==631==
==631== For lists of detected and suppressed errors, rerun with: -s
==631== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
haasini@SSH:~/OSSP_upload/Skill_Week11$

```

Observations:

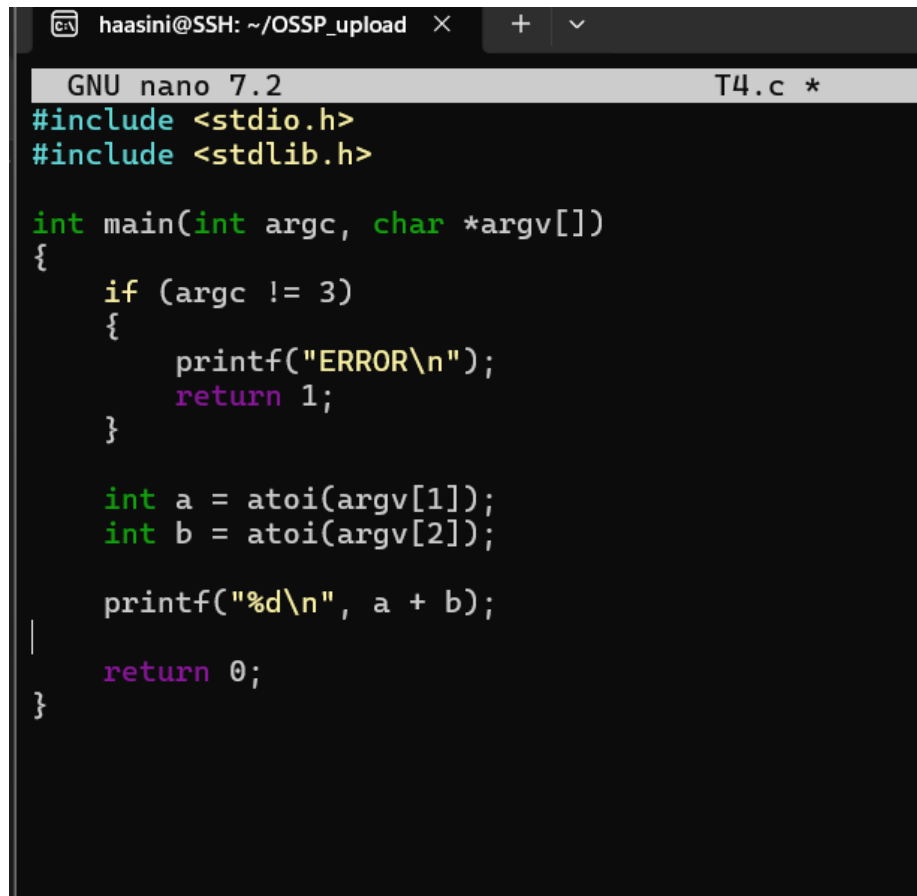
1. `malloc(5 * sizeof(int))` allocates dynamic memory for an array of 5 integers, returning a pointer to the allocated block.
2. The leaking version (`T3_leak.c`) intentionally omits `free()` at the end, causing allocated memory to remain unreleased.
3. Valgrind detects 20 bytes in 1 block as 'definitely lost' in the leak version, confirming memory was allocated but never freed.
4. The HEAP SUMMARY reports total heap usage: 2 allocs, 1 free (leak version), indicating one allocated block was not freed.
5. The LEAK SUMMARY specifies the exact allocation site (`malloc` in `T3_leak.c:6`) where the unreleased memory originated.
6. The fixed version (`T3_fixed.c`) adds `free(numbers)` before return, properly releasing the allocated memory.
7. Valgrind verifies the fixed version shows 'in use at exit: 0 bytes in 0 blocks', confirming complete memory cleanup.
8. The fixed HEAP SUMMARY reports: 2 allocs, 2 frees, demonstrating all allocated blocks were properly freed.
9. The fix verification confirms 'All heap blocks were freed -- no leaks are possible', validating successful memory leak resolution.

Task 4: Automated Testing and Test Verification

Question:

Design test cases, create test scripts, automate execution, verify outputs, compare results, and generate reports.

Source Code:

A screenshot of a terminal window with a dark background. The title bar at the top shows 'haasini@SSH: ~/OSSP_upload' and a tab icon. Below the title bar, the editor header reads 'GNU nano 7.2' on the left and 'T4.c *' on the right. The code is written in C and uses syntax highlighting: keywords like 'int', 'if', 'return', and 'printf' are in green, string literals are in yellow, and variable names and numbers are in white. The code defines a 'main' function that checks the number of command-line arguments. If there are not exactly 3 arguments, it prints 'ERROR\n' and returns 1. Otherwise, it converts the second and third arguments to integers using 'atoi', adds them, and prints the result. The code is enclosed in curly braces for function and conditional blocks.

```
GNU nano 7.2 T4.c *
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("ERROR\n");
        return 1;
    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    printf("%d\n", a + b);

    return 0;
}
```

```

GNU nano 7.2                                test_T4.sh *
#!/bin/bash

PASS=0
FAIL=0

run_test()
{
    TEST_NAME="$1"
    EXPECTED="$2"
    shift 2

    ACTUAL=$(./T4 "$@" )

    if [ "$ACTUAL" = "$EXPECTED" ]; then
        echo "[PASS] $TEST_NAME"
        PASS=$((PASS + 1))
    else
        echo "[FAIL] $TEST_NAME"
        echo "    Expected: $EXPECTED"
        echo "    Actual   : $ACTUAL"
        FAIL=$((FAIL + 1))
    fi
}

echo "=== Automated Test Report ==="

run_test "Positive numbers" "15" 10 5
run_test "Negative numbers" "-8" -3 -5
run_test "Mixed numbers" "5" 10 -5
run_test "Zero values" "0" 0 0
run_test "Missing argument" "ERROR" 5

echo
echo "Passed: $PASS"
echo "Failed: $FAIL"

if [ "$FAIL" -eq 0 ]; then
    echo "Overall Result: ALL TESTS PASSED"
else
    echo "Overall Result: SOME TESTS FAILED"
fi

```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week11$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week11$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week11$ nano test_T4.sh

haasini@SSH:~/OSSP_upload/Skill_Week11$ chmod +x test_T4.sh
haasini@SSH:~/OSSP_upload/Skill_Week11$ ./test_T4.sh
=== Automated Test Report ===
[PASS] Positive numbers
[PASS] Negative numbers
[PASS] Mixed numbers
[PASS] Zero values
[PASS] Missing argument

Passed: 5
Failed: 0
Overall Result: ALL TESTS PASSED

```

Observations:

1. A test program (T4.c) accepts two command-line numeric arguments, converts them with `atoi()`, and prints their sum.
2. The program validates input using `argc != 3`; if the argument count is incorrect, it prints 'ERROR' and returns 1.
3. A bash test script (test_T4.sh) defines a `run_test()` function that executes T4 with specific inputs and compares actual output to expected output.
4. Five test cases are defined in the script: Positive numbers (inputs 10 and 5, expected 15); Negative numbers (inputs -3 and -5, expected -8); Mixed numbers (inputs 10 and -5, expected 5); Zero values (inputs 0 and 0, expected 0); Missing argument (input 5 only, expected ERROR).
5. The `run_test()` function captures actual program output via command substitution and compares it to the stored expected value.
6. A PASS result is recorded when actual output matches expected; the PASS counter increments and the test name is printed with [PASS].
7. A FAIL result is recorded when output does not match; the FAIL counter increments and expected versus actual values are displayed.
8. The final report prints the total Passed and Failed counts.
9. The output confirms Passed: 5, Failed: 0 and 'Overall Result: ALL TESTS PASSED', verifying correct behaviour across all five test cases.